ISDC

BUSINESS ANALYTICS

LIVE PROJECT

RETAIL SALES ANALYSIS

Particulars	Details
Team Member 1	Sanjana Nekkanti – 24BBA7109
Team Member 2	Indira Priyadharshini – 24BBA7090
Team Member 3	Bhuvan – 24BBA7094
University	VIT-AP University
Department	Department of BBA
Course	Business Analytics (Live Project)
Trainer / Faculty	Miss. Jilu Balakrishnan
Project Repository	GitHub repository link to be added after project upload.

VIT-AP UNIVERSITY
VIJAYAWADA, ANDHRA PRADESH, INDIA

Abstract

Retail Sales Analysis is a Business Analytics project designed to organise and examine retail transaction data using Python, pandas, SQL, data visualisation and an interactive dashboard. The project aims to identify sales patterns, product performance, category-wise contribution, payment preferences and changes in sales over time. The workflow includes loading transaction data, cleaning and validating records, storing structured data in SQLite, running analytical queries, preparing visual summaries and presenting the results through a Streamlit dashboard. Any conclusions will be limited to the fields, time period and records available in the selected dataset. Numerical findings must be generated from and verified against the actual project data before final submission.

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Tools and Technologies
6. Methodology
7. Project Structure
8. Results and Observations
9. Dashboard Features
10. How to Run the Project
11. Limitations
12. Future Enhancements
13. Conclusion
14. Project Repository

1. Introduction

Retail businesses generate transaction records whenever customers purchase products. These records can contain details such as invoice number, date, product, category, quantity, unit price, discount, payment method and store or customer information. When these details are stored in spreadsheets or multiple files, it may be difficult to identify sales trends and compare product performance manually.

This project applies business analytics techniques to transform retail transaction records into useful summaries. Python and pandas are used for data preparation, SQLite is used for structured storage, SQL is used for querying, and visualisation tools are used to present patterns in an understandable form. A Streamlit dashboard provides an interactive way to explore the results.

2. Problem Statement

Retail transaction data may contain inconsistent formats, missing values, duplicate entries and large numbers of records. Without systematic analysis, it can be difficult for a business to understand which products or categories contribute most to sales, how sales change over time, and which payment methods are commonly used. The project addresses this need by creating a repeatable process for cleaning, analysing and visualising retail sales data.

3. Objectives

• Load and inspect retail transaction data using Python and pandas.

• Clean the dataset by checking missing values, duplicate records, data types and invalid entries.

• Store the prepared data in an SQLite database for structured access.

• Use SQL and Python to calculate sales totals, quantities, average transaction values and category summaries.

• Analyse product and category performance and examine sales trends over time.

• Create charts and an interactive Streamlit dashboard with filters and data tables.

• Prepare clear findings and recommendations based only on verified results.

4. Dataset Description

The project requires a retail sales transaction dataset in CSV or Excel format. The final analysis will use the actual columns available in the selected file. The following table describes common fields that may be used; fields not present in the source dataset will be excluded rather than assumed.

Field	Purpose / interpretation
Transaction / Invoice ID	Unique reference for a sales transaction or invoice.
Date	Date on which the transaction was recorded; used for time-based analysis.
Product / Item	Name or code identifying the item sold.
Category	Product group used for category-wise comparisons.
Quantity	Number of units recorded in the transaction.
Unit Price	Price per unit, where available.
Sales / Total Amount	Recorded transaction value or calculated quantity multiplied by unit price, where appropriate.
Discount	Discount value or rate, if included in the source data.
Payment Method	Recorded payment type, if available.

Data validation note: the dataset's actual column names, units, date format and definitions must be checked before writing code or SQL queries. A calculated sales value must not replace a source field unless its calculation is clearly documented.

5. Tools and Technologies

Technology	Role in the project
Python	Runs data preparation, analysis and reporting scripts.
Pandas	Loads, cleans, transforms and summarises tabular data.
SQLite	Stores prepared retail data in a local relational database.
SQL	Queries sales, product, category and transaction summaries.
Streamlit	Provides the interactive dashboard interface.
Plotly / Matplotlib	Creates charts and visual summaries.
Visual Studio Code	Used to develop and run the project files.
Git and GitHub	Track project versions and host the source code repository.

6. Methodology

Step	Stage	Work carried out
1	Data loading	Import the source CSV or Excel file into Python using pandas.
2	Data inspection	Review column names, data types, missing values, duplicates and unusual values.
3	Data cleaning	Standardise formats and handle data-quality issues using documented rules.
4	Database storage	Save the cleaned dataset in SQLite tables for repeatable analysis.
5	Querying and analysis	Use SQL and Python to calculate sales totals, quantity, average order value and category/product summaries where supported.
6	Visualisation	Prepare bar, line, pie or other suitable charts to show verified patterns.
7	Dashboard development	Build a Streamlit dashboard with KPI cards, filters, charts and a transaction table.
8	Validation and reporting	Cross-check dashboard totals against SQL outputs and prepare the final report.

7. Project Structure

File / folder	Purpose
app.py / dashboard.py	Main Streamlit dashboard application.
data/	Stores the source dataset or prepared data, subject to data-use permissions.
data_cleaning.py	Data loading, validation and cleaning routines.
analysis.py	Calculates sales and product/category summaries.
database.py	Creates and manages SQLite database tables.
queries.sql	Contains SQL statements used for analysis.
visualizations.py	Generates charts for sales trends and comparisons.
requirements.txt	Lists Python package dependencies.
README.md	Project overview, setup instructions and repository information.
reports/ and charts/	Stores verified summary outputs and generated visualisations.

8. Results and Observations

The project is designed to report key measures such as total sales, total units sold, number of transactions, average transaction value, top-performing products and category-wise sales contribution. Time-based analysis may be included when the dataset contains valid date information.

No numerical results are stated in this report because the actual retail dataset and generated analysis outputs have not been supplied with the project brief. After running the code, all KPI values, rankings and observations must be taken from the current output files and cross-checked against the database. The team should not present illustrative values as actual findings.

Measure	How it will be interpreted
Total Sales	Sum of the valid sales amount field or documented calculated transaction values.
Total Quantity	Sum of units sold, if quantity is available.
Number of Transactions	Count of unique invoice or transaction IDs, where available.
Average Transaction Value	Total sales divided by the number of valid transactions, where both measures are available.
Top Products / Categories	Products or categories ranked using verified sales or quantity measures.
Sales Trend	Sales aggregated by day, month or another valid time period.

9. Dashboard Features

• KPI cards for total sales, quantity sold and transaction count, depending on available fields.

• Bar chart to compare sales by product or category.

• Line chart to examine sales over time when valid date data exists.

• Pie or donut chart to display category or payment-method distribution when suitable.

• Sidebar filters for date range, category, product or payment method, where available.

• Transaction table for reviewing the underlying records.

• CSV download option for exporting filtered data.

The dashboard is an exploratory interface. Values displayed will depend on the dataset loaded and the filters selected by the viewer.

10. How to Run the Project

Open the project folder in Visual Studio Code and select Terminal → New Terminal. Create a virtual environment if required, install the dependencies and run the dashboard after the project files have been prepared.

```
python -m venv .venv
.venv\\Scripts\\activate (Windows)
source .venv/bin/activate (macOS / Linux)
python -m pip install -r requirements.txt
streamlit run dashboard.py
```

The exact run command may change depending on the final filenames used by the team. Keep the terminal open while using the dashboard and open the local URL displayed in the terminal.

11. Limitations

• The analysis depends on the completeness, accuracy and definitions of the source retail dataset.

• A limited dataset may not represent all customers, stores, products or seasons.

• Missing values, duplicate transactions and inconsistent product labels can affect the results.

• Sales totals may be affected by tax, returns, discounts or cancellations if these are not represented clearly.

• Customer-level analysis should only be performed if suitable fields are available and privacy requirements are respected.

• All reported statistics must be checked against the latest generated outputs before academic submission.

12. Future Enhancements

• Add automated checks for missing values, duplicate transactions and invalid prices or quantities.

• Create a data dictionary documenting every field and its unit.

• Add store-wise or region-wise comparisons if location fields are available.

• Include profit and margin analysis if cost data is available.

• Improve dashboard interactivity with additional filters and clear chart explanations.

• Automate report export and add tests to validate analytical calculations.

13. Conclusion

Retail Sales Analysis demonstrates a practical Business Analytics workflow for organising transaction data, preparing it for analysis, storing it in SQLite, querying it with SQL, creating visualisations and presenting the results through a Streamlit dashboard. The project can help users explore sales performance, product and category contribution, and time-based patterns where the source data supports them. The reliability of the conclusions depends on data quality, correct calculations and validation of all reported outputs.

14. Project Repository

GitHub repository: To be added after the team creates the repository and uploads the final project files. The repository should contain the source code, requirements.txt, README.md, SQL queries and

permitted sample data or instructions for obtaining the dataset.

Suggested repository name: retail-sales-analysis-live-project

Team: Sanjana Nekkanti (24BBA7109), Indira Priyadharshini (24BBA7090), Bhuvan (24BBA7094).